Búsqueda Heurística

A lo largo de esta práctica se va a implementar el algoritmo conocido como ‘A estrella’. Es un método de búsqueda informada que permite encontrar el camino más corto desde un punto de inicio hasta un objetivo dentro de un espacio de búsqueda, utilizando una heurística para mejorar la eficiencia.

En nuestro caso se desarrolla sobre una cuadrícula o tablero de dimensiones 2D. Está basado en la exploración de nodos vecinos tal y como hace el algoritmo Dijkstra, pero al cual se le añade una función de heurística que sirve para guiar por el camino que es más probable y por tanto es más eficiente. Este algoritmo asegura encontrar el camino más corto siempre que exista uno.

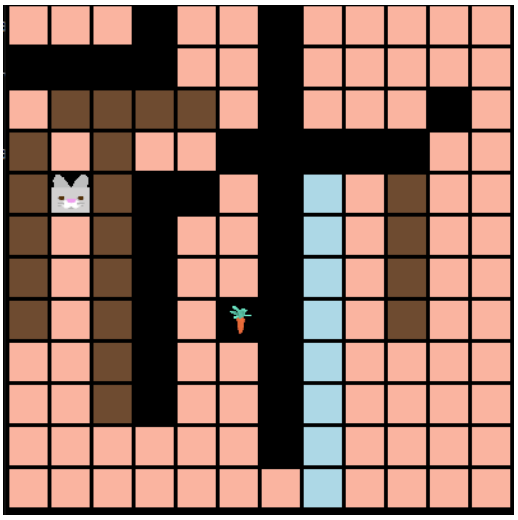


Ilustración 1 (Tablero)

Para que quede más claro, en la *Ilustración 1* podemos ver una imagen de lo que podría ser un tablero de juego, la figura del conejo y de la zanahoria representan la casilla de inicio y la de final respectivamente. Los cambios en el color de las casillas representan los diferentes pesos que tendría pasar por ellas (asignados en calorías), o la imposibilidad de atravesarlas, como en el caso de las de color negro.

Entorno de trabajo

Todo el desarrollo de código del algoritmo se ha desarrollado en lenguaje Python, usando la aplicación de Visual Studio Code.

Para la ejecución del código se ha usado el programa Miniconda, el cual nos permite instalar diferentes librerías de Python mediante la instrucción ‘pip’, necesario para ejecutar la interfaz gráfica.

Movimientos permitidos

Se establecen en 8 posibles direcciones tal y como se muestra en la *Ilustración 2*.

Los movimientos verticales u horizontales tienen un coste g=1, mientras que los movimientos diagonales tienen un coste g=1,5. Esto lo veremos más adelante.

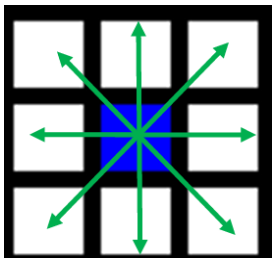


Ilustración 2
(movimientos aceptados)

Definición de la función de coste

La función de coste 'f' es la métrica con la cual el algoritmo decide cuales son los mejores nodos para construir el camino. La función tiene la forma: $f(c) = g + h$

- 'g' representa el coste real del camino a seguir siendo el coste total el sumatorio desde el origen hasta el objetivo.
- 'h' hace referencia a la función heurística y se calcula para cada celda que se explora.
-

Explicación del algoritmo

El proceso comienza con la exploración de vecinos, donde desde el nodo inicial se expanden los 8 vecinos adyacentes en una cuadrícula. Para cada uno de estos vecinos, se calcula la función de costo f(n), que consta de dos componentes como hemos visto anteriormente:

1. **g(n)**: El costo real acumulado desde el nodo de inicio hasta el nodo actual.
2. **h(n)**: Una estimación heurística del costo restante hasta el destino.

El siguiente nodo a explorar será aquel con el menor valor de f(n), ya que representa la mejor estimación de la ruta óptima. Todos los nodos explorados se almacenan en una lista para evitar ciclos y volver a visitar nodos innecesariamente.

Si en algún momento uno de los nodos explorados no nos permite continuar con el camino se vuelve hacia atrás hasta un nodo el cual sí que lo permita o tenga mejor estimación.

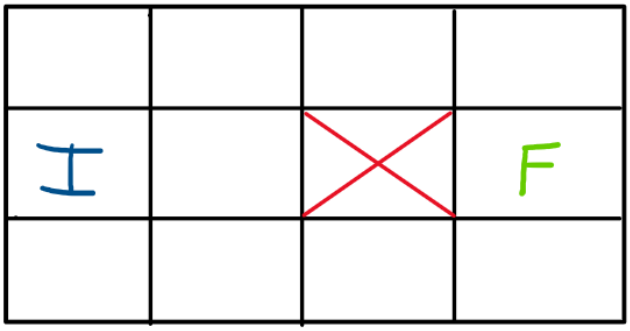
El algoritmo continúa avanzando de un nodo a otro siguiendo la estrategia de seleccionar el nodo con menor costo estimado, siempre verificando que sea transitable. Una vez que se alcanza el nodo objetivo, se reconstruye el camino óptimo retrocediendo desde el destino hasta el inicio, siguiendo los nodos denominados como "father". Finalmente, la ruta encontrada puede representarse visualmente.

Resolución de un problema pequeño

Con el fin de probar el funcionamiento del algoritmo en su caso más básico se ha creado un mapa básico con una solución sencilla.

En este caso se ha empleado una heurística trivial, en la que no se tiene en cuenta ningún tipo de distancia hasta el objetivo. Únicamente se van explorando caminos vecinos hasta llegar al objetivo final (si lo hubiera). Es un método de expansión donde se exploran muchos más nodos que empleando medidas de distancia.

Para empezar vamos a ver la distribución del mapa a resolver.



En el mapa podemos diferenciar una casilla de inicio (azul), una casilla de final (verde) y una casilla con una x, la cual es intransitable.

Ilustración 3

En una primera expansión de exploración de vecinos añade a la lista las celdas de color azul (ilustración 4).

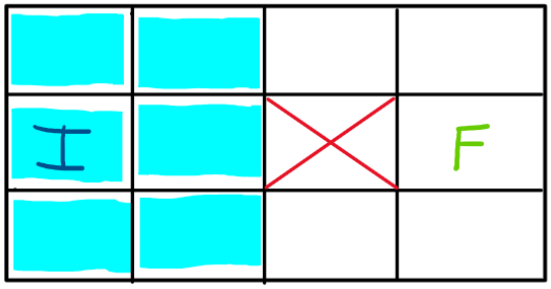


Ilustración 4

Ahora vemos que se ha movido a la casilla amarilla para tratar de explorar casillas vecinas. Como las de alrededor ya las ha explorado se volverá a mover a otra posición.

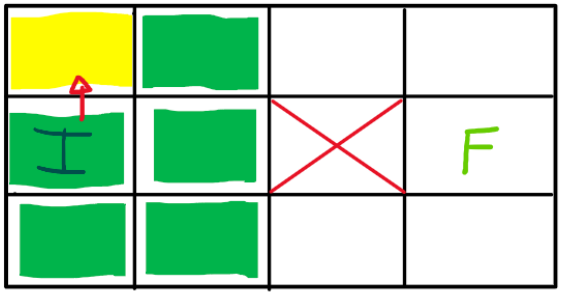


Ilustración 5

En este caso trata de explorar de nuevo el nodo de inicio pero como ya es un nodo explorado lo omite y pasa al siguiente.

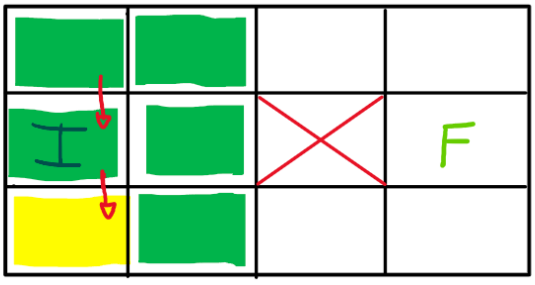


Ilustración 6

En este caso hemos ido más adelante y vemos que se coloca en la casilla 1,1 (amarillo) para seguir explorando los nodos de la derecha, detecta que el que tiene exactamente a la derecha es zona no transitable.

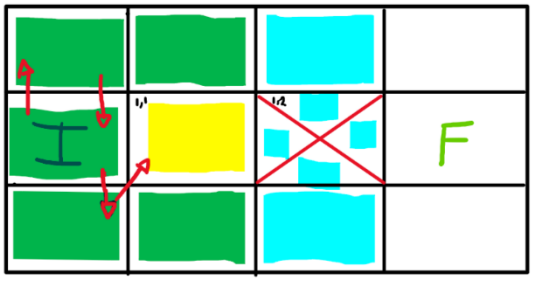


Ilustración 7

En este paso se puede apreciar con las flechas rojas el camino que va siguiendo y como usa todos los nodos posibles para explorar nuevos nodos (si lo hubiera) o para encontrar el final. Además añade a la lista frontera las dos celdas azules.

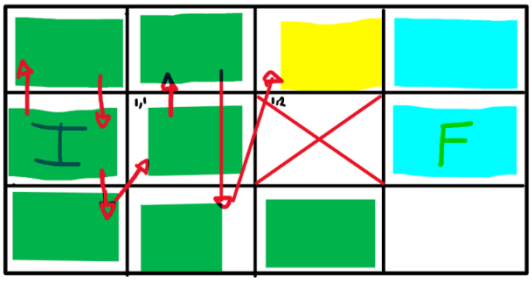


Ilustración 8

Al final va recorriendo todos los nodos que consigue explorar uno a uno hasta posicionarse por fin en el nodo objetivo.

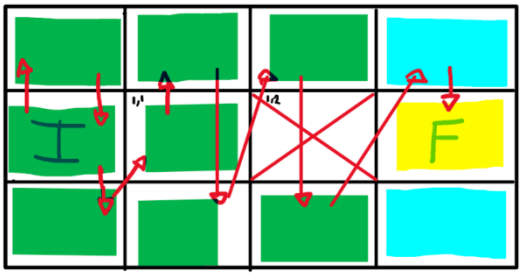


Ilustración 9

Como esta heurística no tiene una optimalidad no puede dirigir su camino en una dirección concreta y acaba explorando todos los nodos de los que dispone hasta que logra encontrar el camino.

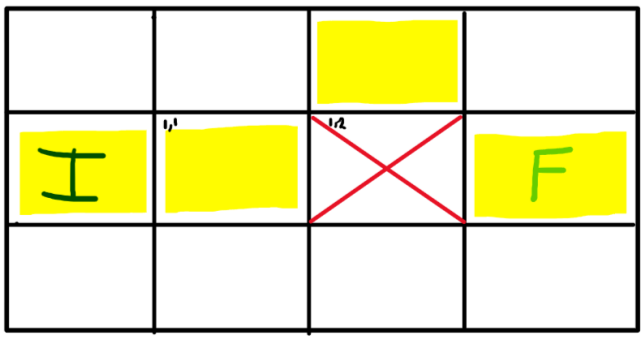


Ilustración 10

Análisis de heurísticas

Una heurística nos da una estimación de coste para llegar desde el nodo actual hasta el nodo objetivo. Esta estimación ha de ser optimista y admisible, esto quiere decir que el valor que se obtenga ha de ser menor que la distancia real hasta el objetivo.

Cuanto mejor sea la heurística que se implemente, menor será el número de nodos explorados y por tanto más eficiente.

Las heurísticas que se han implementado son las siguientes:

Distancia manhattan:

$$h = |dx - ox| + |dy - oy|$$

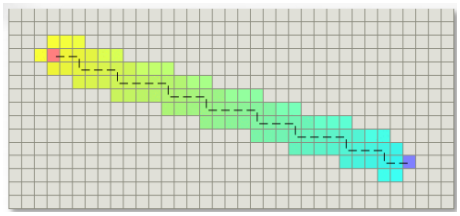


Ilustración 11 (Manhattan)

Trivial:

$$h = 0$$

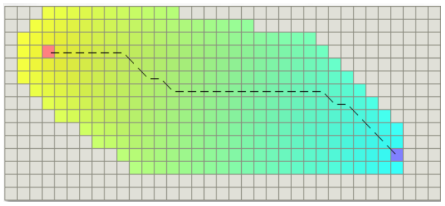


Ilustración 13 (Trivial)

Distancia diagonal:

$$h = max(|n.-goal.x|, abs|n.-goal.y|)$$

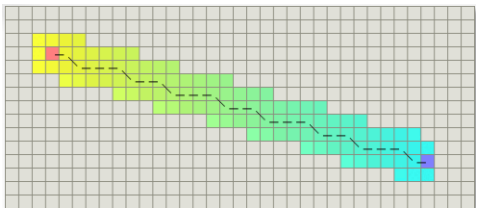


Ilustración 12 (Diagonal)

Distancia euclídea:

$$h = \sqrt{(dx - ox)^2 + (dy - oy)^2}$$

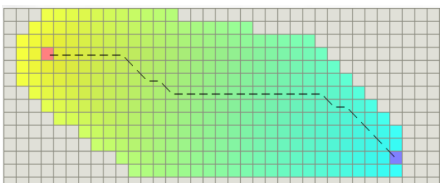


Ilustración 14 (Euclídea)

**Las ilustraciones 5 y 6 son iguales pero el concepto es el mismo, en la heurística trivial se exploran gran cantidad de nodos hasta llegar al objetivo, más de los que pueden aparecer en la imagen.*

Con el fin de poder ver realmente el funcionamiento de las distintas heurísticas se han generado los siguientes gráficos en los que aparece el número de nodos explorados en cada uno de los mapas de prueba con diferentes dificultades.

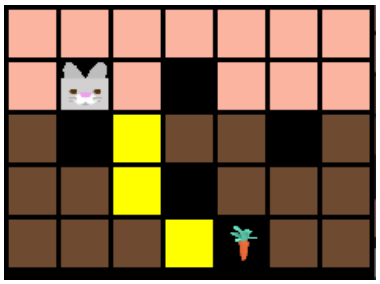


Ilustración 16 (Mapa 1)

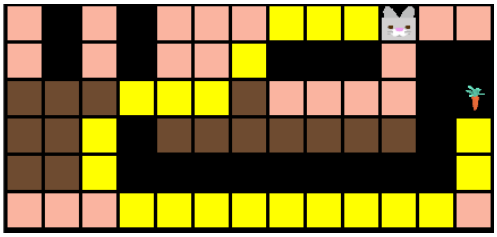


Ilustración 15 (Mapa 2)

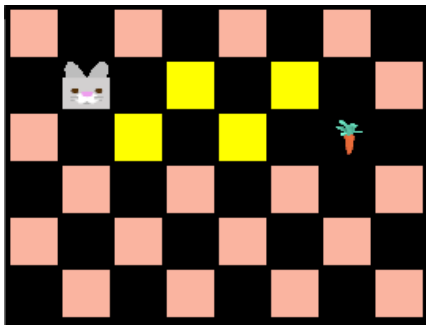


Ilustración 18 (Mapa 3)

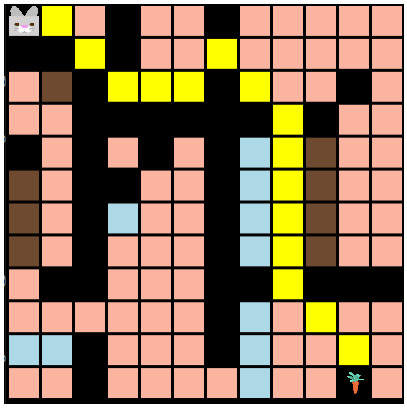


Ilustración 17 (Mapa 4)

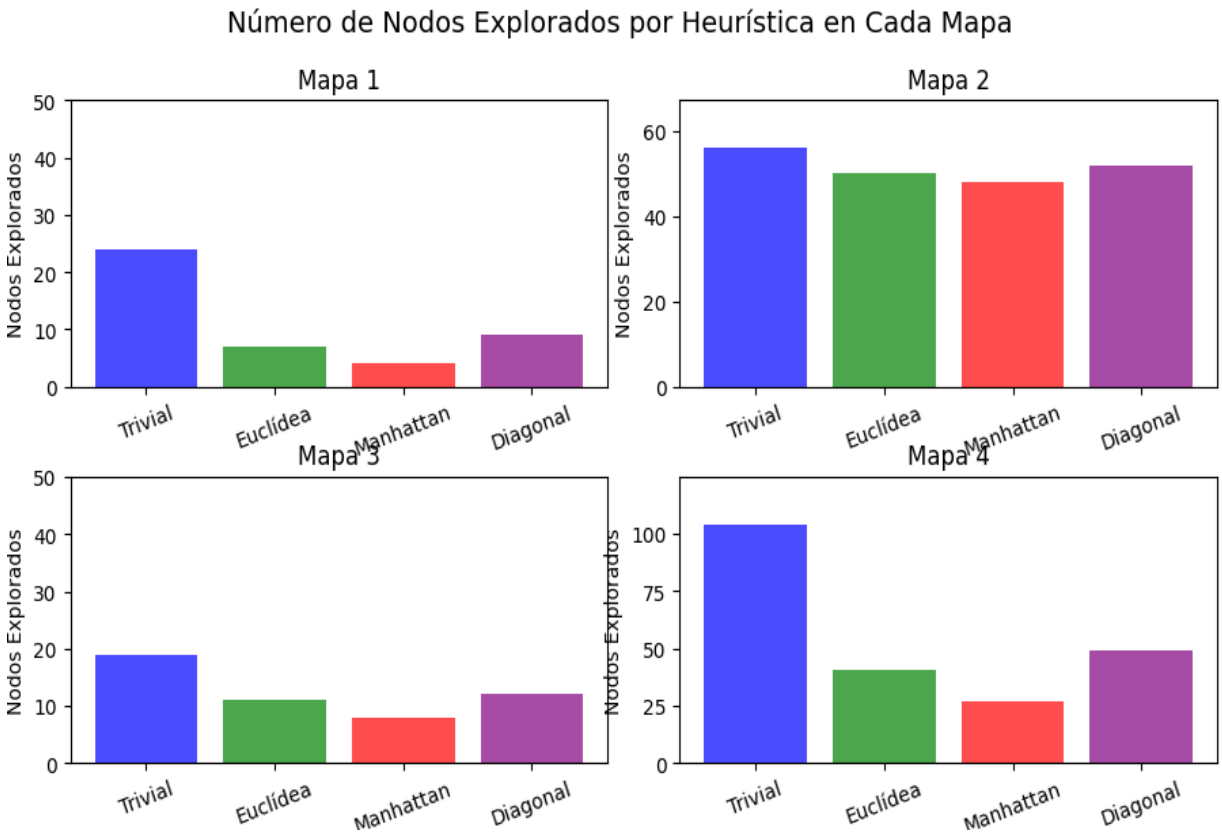


Ilustración 19 (Gráfico)

Como se puede apreciar en el gráfico, en cada uno de los casos la heurística más eficiente siempre acaba siendo la que emplea el cálculo de la distancia Manhattan. Es la que logra llegar al destino explorando menos nodos.

Un elemento que destacar es que aunque todas las heurísticas encuentren siempre el camino con el menor coste posible, suelen cambiar algún nodo del recorrido; es decir, no siguen el mismo camino, pero sí que tienen el mismo coste final. Esto por la forma en la que cada heurística determina si el nodo el cual se va a explorar tiene una mejor estimación de llegada que algún otro nodo vecino.

Caso A estrella ε

En este caso el algoritmo funciona de la misma manera pero se añade una relajación de la restricción de optimalidad. Es decir, ahora los nodos que puede escoger para seguir ya no será el que menos coste tenga sino un subconjunto el cual esté dentro de un rango proveniente de un factor épsilon; definido por la siguiente ecuación:

$$Lf = \{n \mid f(n) \leq (1+\varepsilon)\min(f(m): m \text{ in Frontera})\}$$

Ecuación 1 (astar épsilon)

Todos los nodos cuya distancia hasta el objetivo sea menor de este rango pueden ser ahora parte de la solución y por eso también hay que incluir otra función de coste, asociada ahora al tipo de camino y no a la dirección de movimiento.

Esta función hace uso de las calorías y van directamente asociadas a cada celda del mapa.

Calorías según el tipo de camino

En cada mapa podemos encontrar 4 tipos de casilla diferenciadas por colores:

- Negro: casilla no transitable por la cual no se puede crear un camino.
- Naranja: valle de coste 2 cal.
- Azul: pantano de coste 4 cal
- Marrón: montaña de coste 6 cal.

Transitar por cada casilla añade las calorías que están definidas y el objetivo es encontrar el camino más corto y con menor gasto de calorías.

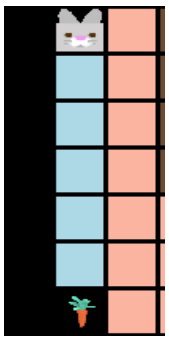


Ilustración 20

En esta disposición (Ilustración 11) podemos ver como el camino es todo de cuadrículas de tipo pantano. Ahora veamos el camino que sigue el algoritmo sin la relajación de optimalidad.

En la Ilustración 12 vemos la solución que nos entregaría el algoritmo Astar básico aplicando cualquier tipo de heurística sin recurrir al peso de cada celda.

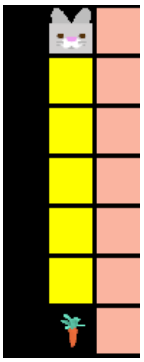
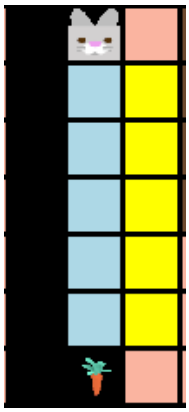


Ilustración 21



En la Ilustración 13 vemos un ejemplo del uso del Astar con la relajación de la optimalidad, en la que se proporciona un valor de épsilon de 0,3. En este caso el algoritmo determina que el camino llano (con ponderación 2 cal) es mucho mejor que el camino tipo pantano (con ponderación 4 cal), y por tanto escoge seguir ese camino.

Ilustración 22

Lo que se consigue haciendo uso de este algoritmo modificado es que sacrificando la longitud del camino (su coste), podemos llegar al punto de destino de forma más barata (con menos calorías).

Para entenderlo podemos hacer una similitud con una carretera. Puede darse el caso que para llegar a un lugar existan 2 caminos posibles, uno por autovía y otro por carreteras secundarias. Probablemente uno de los caminos sea más rápido pero el otro puede requerir un menor gasto de combustible.

Resultados en función de ϵ

Vamos a analizar el mapa: mapa_traza_eps.txt.

En primer lugar podemos ver su distribución de celdas por colores donde cada una tiene un peso asociado correspondiente, como hemos mencionado anteriormente.

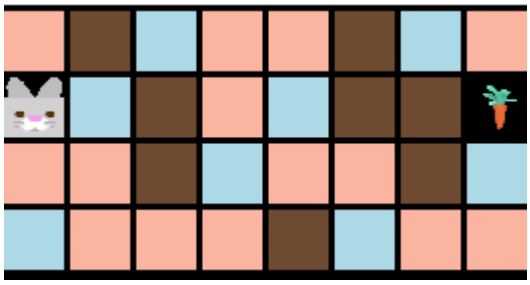
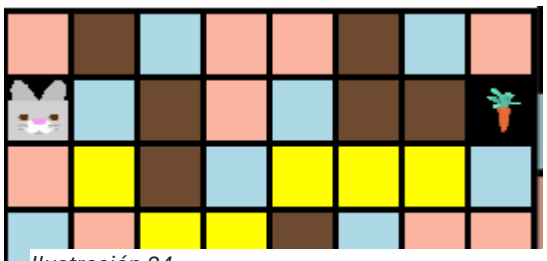


Ilustración 23

Con esta disposición y con un valor $\epsilon = 0.3$ obtenemos el siguiente resultado:



Se puede apreciar la diferencia que hay al trazar un camino implementando esta modificación.

Ilustración 24

Si por el contrario, ϵ fuera 0 el camino a seguir sería una línea recta como se puede ver en la *ilustración 25*.

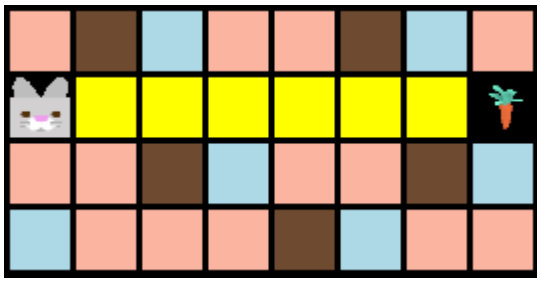


Ilustración 25

Ahora vamos a modificar el valor de épsilon, para la siguiente prueba usaremos el mismo mapa y la misma disposición de casillas inicio/final. Usando $\epsilon = 0.15$ el resultado es el siguiente:

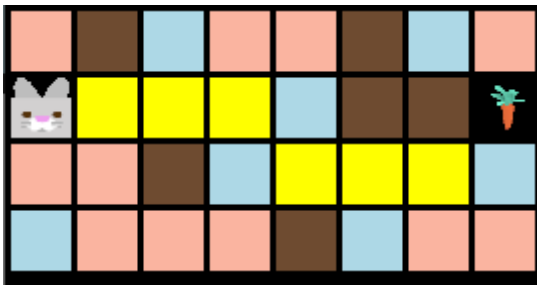


Ilustración 26

Podemos apreciar que el resultado ha cambiado. Esto se debe a la introducción de la ecuación que crea la sublista a partir de la cual se van a ir escogiendo las casillas.

Al reducir significativamente esta cota no permite tanta desviación en el camino y por tanto vemos este resultado, el cual ahora tiene un coste de 24 calorías cuando en la *ilustración 24* tenía únicamente 18 calorías.

Vamos a probar distintos valores en el "mapa5.txt" para ver como varía el resultado según ajustamos el valor ϵ.

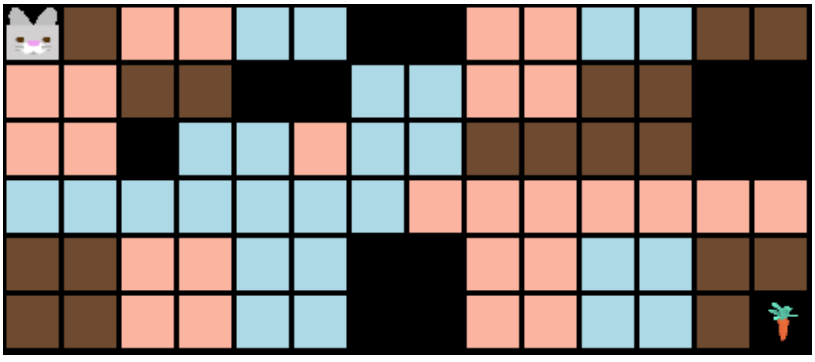


Ilustración 27

Vamos a ir viendo los resultados que van apareciendo.

Valor $\epsilon = 0.7$

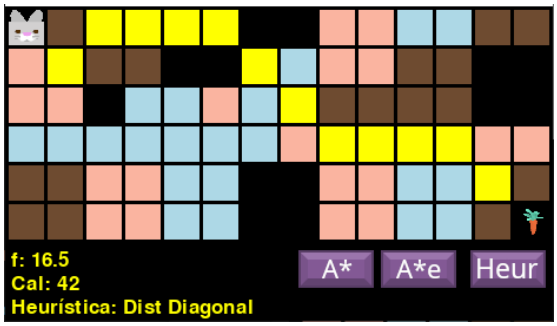


Ilustración 28

Valor $\epsilon = 0.4$

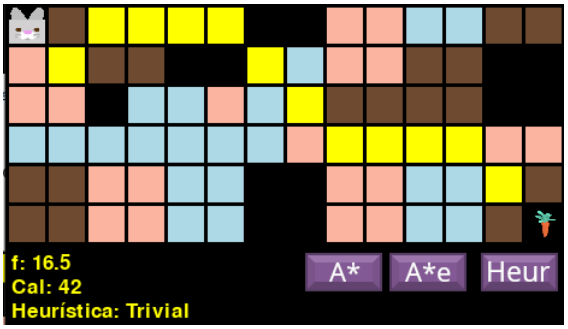


Ilustración 29

Valor $\epsilon = 0.25$

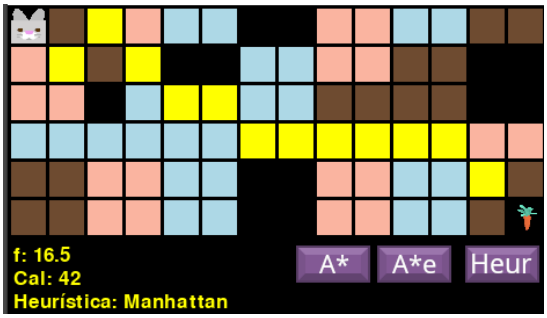


Ilustración 30

Valor $\epsilon = 0.07$

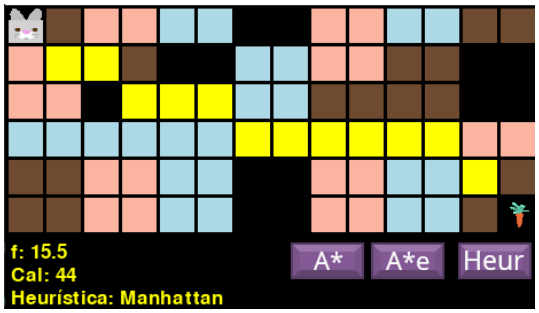


Ilustración 31

En el caso de este mapa vemos que no hay mucha diferencia en los resultados obtenidos. Si disminuimos drásticamente el valor, si que podemos ver que el resultado se aleja de la solución óptima entre camino más corto y con menor coste.

Selección del valor épsilon

El valor de épsilon que usemos a la hora de hacer uso del algoritmo con la relajación de optimalidad va a ser clave a la hora de obtener unos buenos resultados.

Si este valor es muy pequeño puede que los nodos que estén dentro del subconjunto de celdas posibles sean muy reducidos, o incluso unitario. Esto conlleva a que no se pueda elegir un camino con un menor gasto de calorías que el esperado. Es decir, que afecta directamente al margen de maniobra el cual se permite para elegir mejores caminos.

Por otro lado, si el valor proporcionado es demasiado grande, el camino resultado se va a alejar de la solución óptima coste de camino/calorías.

Según la experimentación, en mapas **abiertos** funcionan mejor los valores de épsilon bajos ya que no tiene que buscar rutas alternativas por caminos cerrados y puede centrarse en ir directamente al objetivo.

En mapas **cerrados** o con más casillas intransitables si épsilon es un valor muy bajo el algoritmo se vuelve rígido y descarta caminos con mejores consumos calóricos. Se pueden usar valores aproximados de 0.5 para mejorar el camino.

En cuanto a las posiciones de inicio-final, si estas se encuentran en diagonal, con valores de 0.3 y junto con una heurística Euclidiana da mejores resultados en cuanto a nodos explorados.

Si estas posiciones son horizontales o verticales, la distancia Manhattan es la mejor opción con menor gasto de calorías y nodos explorados, el valor de épsilon puede oscilar entre 0.2 y 0.4.

En conclusión, si queremos un resultado de camino más corto debemos escoger un valor de épsilon menor, pero si lo que nos importa es el gasto calórico podemos relajar la restricción aumentando este valor hasta aproximadamente 0.5.

Un buen equilibrio entre ambos puede rondar el valor de 0.3, a partir de ahí siempre se suele hacer un camino eficiente.

Bibliografía

Heurísticas:

http://www.lcc.uma.es/~monte/MaterialDocente/SistemasInteligentes/2SesionLab.pdf

Explicación del algoritmo:

https://www.toolify.ai/es/ai-news-es/algoritmo-aestrella-encuentra-la-ruta-ptima-en-un-laberinto-1328513

https://www.youtube.com/watch?v=X-5JMScsZ14